

Aim: Implementing sorting algorithm i.e Bubble sort, Insertion sort, Quick sort, Merge sort, Selection sort.

1. Imagine you are a librarian who has to organize a collection of ancient books discovery in a hidden library. Each of these books is marked by a numeric identifier but the books are all jumbled to unsure that they are catalog properly you decide to use the ancient art of bubble sorting to arrange them in a numerical order. WAP in C++ to help with this catalog.

CODE:

```
#include <iostream>
using namespace std;
int sortBook(int n){
    int book[n];
    cout<<"Enter the ancient book special identifier:";
    for(int i=0;i<n;i++){
        cin>>book[i];
    }
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(book[j]>book[j+1]){
                int t=book[j];
                book[j]=book[j+1];
                book[j+1]=t;
            }
        }
    }
    cout<<"Arrangement of ancient book using the ancient art:";
    for(int i=0;i<n;i++){
        cout<<book[i]<<" ";
    }
    return 0;
}
int main(){
    int n;
    cout<<"Enter the total number of ancient books you recieved:";
    cin>>n;
    sortBook(n);
    return 0;
}
```

OUTPUT:

```
Enter the total number of ancient books you recieved:5
Enter the ancient book special identifier:201 501 301 401 101
Arrangement of ancient book using the ancient art:101 201 301 401 501
-----
Process exited after 9.582 seconds with return value 0
Press any key to continue . . . |
```

2. There is a mystical forest inhabited by talking animals. Unfortunately, the animals are captured and taken to the circus where they have to show their skills one by one. The ring master is a cruel man and only wants them to show their talents only in reverse alphabetical order. You, being an animal lover, want to help these animals achieve this order in order to escape the wrath of the ring master using insertion sort.

CODE:

```
#include <iostream>
#include <cstring>
using namespace std;
int sortArray(int n, string an[]){
    int j;
    for(int i=1; i<n; i++){
        string t=an[i];
        j=i-1;
        while(j>=0 && an[j]>t){
            an[j+1]=an[j];
            j=j-1;
        }
        an[j+1]=t;
    }
    cout<<"Mythical Beast in reverse order:";
    for(int i=n-1; i>=0; i--){
        cout<<an[i]<<" ";
    }
    cout<<endl;
    return 0;
}
int main(){
    int n;
    cout<<"How many animals are captured in a circus:";
    cin>>n;
    string an[n];
    cout<<"Give the names of the captured animals:";
    for(int i=0; i<n; i++){
        cin>>an[i];
    }
    sortArray(n, an);
    cout<<"Congrats!, You saved the Mythical Beast.";
    return 0;
}
```

OUTPUT:

```
How many animals are captured in a circus:5
Give the names of the captured animals:Veldora Pegasus Shenron Griffin Behemoth
Mythical Beast in reverse order:Veldora Shenron Pegasus Griffin Behemoth
Congrats!, You saved the Mythical Beast.
-----
Process exited after 55.26 seconds with return value 0
Press any key to continue . . . |
```

3. Once upon a time in a kingdom of Rimuru, queen Elizabeth and ten magical crown jewels each jewel glowed with a unique brightness measured in lumens. The queen wants to wear for the grand ball but only they are arranged from the dulllest to the brightness forming a shimmering gradient as moon light hits them. Queen Elizabeth has summoned you and commanded you to arrange the jewel in increasing order of brightness you have to use selection sort algorithm to help the queen.

CODE:

```
#include <iostream>
#include <cstring>
using namespace std;
int sortArray(int lum[]){
    int i,j;
    for(i=0;i<10-1;i++){
        for(j=i+1;j<10;j++){
            if(lum[i]>lum[j]){
                int t=lum[i];
                lum[i]=lum[j];
                lum[j]=t;
            }
        }
    }
    cout<<"The crown is arranged from dulllest to brightest:";
    for(i=0;i<10;i++){
        cout<<lum[i]<<" ";
    }
    return 0;
}
int main(){
    int lum[10];
    cout<<"For 10 crown jewel that the Queen Elizabeth has..."<<endl;
    cout<<"Enter the lumens for each crown:";
    for(int i=0;i<10;i++){
        cin>>lum[i];
    }
    sortArray(lum);
    return 0;
}
```

OUTPUT:

```
For 10 crown jewel that the Queen Elizabeth has...
Enter the lumens for each crown:90 85 70 10 40 30 50 20 50 110
The crown is arranged from dulllest to brightest:10 20 30 40 50 50 70 85 90 110
-----
Process exited after 110.7 seconds with return value 0
Press any key to continue . . . |
```

4.Aruna runs a successful online Yarn store and often receive large bulk orders from different clients. Each clients gives a list of quantities for a different Yarn category to be delivered she as combined all order quantities and wants to sort them from the largest quantities to the smallest. Help her using the merge sort algorithm.

CODE:

```
#include <iostream>
#include <cstring>
using namespace std;
void merge(int yarn[],int left,int mid,int right){
    int n1=mid-left+1;
    int n2=right-mid;
    int L[n1],R[n2];
    for(int i=0;i<n1;i++){
        L[i]=yarn[left+i];
    }
    for(int j=0;j<n2;j++){
        R[j]=yarn[mid+j+1];
    }
    int i=0,j=0,k=left;
    while(i<n1 && j<n2){
        if(L[i]<=R[j]){
            yarn[k++]=L[i++];
        }
        else{
            yarn[k++]=R[j++];
        }
    }
    while(i<n1){
        yarn[k++]=L[i++];
    }
    while(j<n2){
        yarn[k++]=R[j++];
    }
}
void mergeSort(int yarn[],int left,int right){
    if(left>=right){
        return;
    }
    int mid=left+(right-left)/2;
    mergeSort(yarn,left,mid);
    mergeSort(yarn,mid+1,right);
    merge(yarn,left,mid,right);
}
int main(){
    int n;
    cout<<"Enter total number of categories for Yarn:";
    cin>>n;
    string y[n];
```

```

        cout<<"Enter Yarn categories(like Cotton, Nylon, etc):";
        for(int
        i=0;i<n;i++){
            cin>>y[i];
        }
        cout<<"Your Yarn catelog:";
        for(int i=0;i<n;i++){
            cout<<y[i]<<" ";
        }
        cout<<"\nEnter the quantities for each category:";
        int yarn[n];
        for(int i=0;i<n;i++){
            cin>>yarn[i];
        }
        mergeSort(yarn, 0, n-1);
        cout<<"Your order for different yarn categories i.e from largest to smallest:";
        for(int i=n-1;i>=0;i--){
            cout<<yarn[i]<<" ";
        }
        return 0;
    }
}

```

OUTPUT:

```

Enter total number of categories for Yarn:5
Enter Yarn categories(like Cotton, Nylon, etc):Cotton Silk Linen Nylon Wool
Your Yarn catelog:Cotton Silk Linen Nylon Wool
Enter the quantities for each category:200 600 1000 800 5000
Your order for different yarn categories i.e from largest to smallest:5000 1000 800 600 200
-----
Process exited after 30.01 seconds with return value 0
Press any key to continue . . . |

```

5.To arrange a list of student names in ascending order using quick sort.**CODE:**

```

#include<iostream>
#include <cstring>
using namespace std;
void swap(string &a,string &b){
    string t=a;
    a=b;
    b=t;
}
int partition(string std[],int low,int high){
    string pivot=std[high];
    int i=low-1;
    for(int j=low;j<=high-1;j++){
        if(std[j]<pivot){
            i++;
            swap(std[i],std[j]);
        }
    }
    swap(std[i+1],std[high]);
}

```

```
        return i+1;
    }
    void quickSort(string std[],int low,int high){
        if(low<high){
            int p=partition(std,low,high);
            quickSort(std,low,p-1);
            quickSort(std,p+1,high);
        }
    }
    int main(){
        int n;
        cout<<"Enter the number of students:";
        cin>>n;
        string std[n];
        cout<<"Enter the names of the students:";
        for(int i=0;i<n;i++){
            cin>>std[i];
        }
        quickSort(std,0,n-1);
        cout<<"Names of student in reverse order:";
        for(int i=0;i<n;i++){
            cout<<std[i]<<" ";
        }
        return 0;
    }
```

OUTPUT:

```
Enter the number of students:5
Enter the names of the students:Jay Rohit Aayush Zameer Aditya
Names of student in reverse order:Aayush Aditya Jay Rohit Zameer
-----
Process exited after 13.66 seconds with return value 0
Press any key to continue . . . |
```